arXiv:2610.34938v1 · cs.SE · 2026-10-26 | HarnessRSI Daily · Reel 第五期

复盘改动写进 harness，下一个任务就会照做吗？ #34938

逐条迁移评审 Christoph Treude（新加坡管理大学）· Sebastian Baltes（海德堡大学） How Do Coding-Agent Harnesses Self-Evolve?

让 Claude Code 用 `/retro` 复盘自己的会话、提出 harness 改动；作者把 12 条改动**一次一条**迁移到同仓库的后续 issue，新容器里各跑 5 次 原版 / 改后。在 7 个有信息的迁移事件中，只有改动**覆盖到会话实际遇到的实例**的 2 条消除了目标动作；1 条减轻，4 条没有减少，还有改动在无关 issue 上增加了工作。

对话 | 改动 24 | 迁移检查 13 | 标注 120 条轨迹

01 问题：写进文件不等于改变行为

编码代理的 harness（CLAUDE.md 等上下文文件、hook、lint/CI 配置）可以在会话后自我修改，留给同仓库的下一次会话用。“改动有没有帮助”要拆成一条链，任一环断开，改动就是摆设：

已实施 改动写入允许的文件 → **已加载** 进入会话上下文 → **已查阅** 读了文件 / 跑了命令 → **动作改变** 不再做目标动作 → **任务结果** 另行记录

本文把“重复工作”操作化为**复现（recurrence）**：后续会话是否又做了该改动想防止的那个具体动作，例如在没装 pytest 的仓库先跑 pytest、或对未修改的代码重跑测试来确认某个失败是否原本就有（基线比对）。这是行为度量，不判断该动作是否多余，也不等于任务做得更好。

迁移检查里转移过去的上下文文件在每个改后会话都被加载，通常也被查阅（表 II 查阅列）；断点集中在“动作改变”这一环。

02 研究动作：一次只迁一条改动

- **30 个会话** SWE-bench Verified；Claude Code 2.1.295 + Opus 5.5
- **125 条提议** 每次复盘 2–6 条，双 LLM 编码 + 一作裁决
- **24 条实施** 回到原会话、原容器实施
- **12 条改动** 11 条可在别的 issue 上评估 + 1 条作者写的对照
- **13 个迁移事件** 同仓库另选 issue，确认环境能复现目标问题
- **7 个有信息** 原版 5 次中 ≥3 次遇到目标问题（预设）

每个事件：**原版 harness × 5** 与 **只多这一条改动 × 5**，各在新 Docker 容器、同一任务提示下运行；3 条改动另加无关 issue 作对照，看是否凭空增加验证工作。

CLAUDE.md（SymPy 例；按 §III 文字复述，非原文件）

```
+ 测试用 bin/test 运行；容器里没装 pytest
+ 结束前跑 CI 的 lint 检查（bin/test quality 等）
```

其余 6 个事件：5 个原版会话几乎没遇到问题；作者写的 pre-commit 对照从未被读或运行。

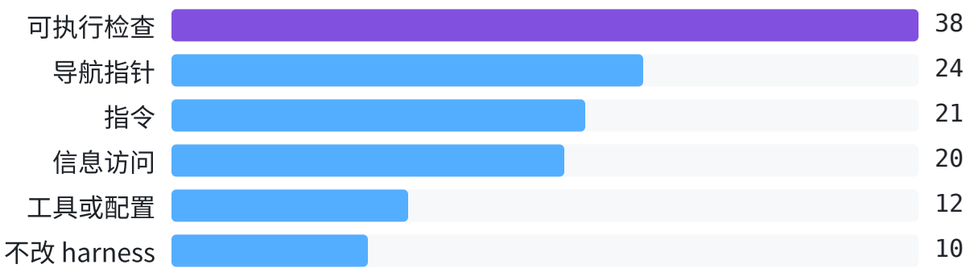
03 · 表 II 重绘 主结果：7 个有信息的迁移事件，原版 vs 改后各 5 个会话

● 该会话做了目标动作（复现；对照行 = 做了基线比对） ○ 没做 ⟲ 剔除（认证错误结束，原文已披露） 分组为作者在标注后按“改动是否覆盖会话实际遇到的实例”所分（事后分组）

仓库	改动	原版 harness	改后 harness	查阅	结论
✓ 改动覆盖了会话实际遇到的实例					
SymPy	CLAUDE.md 一行指明 bin/test	●●●●● 5/5	○○○○○ 0/5	5/5	消除 首次 pytest 尝试全部消失
Astropy	known-failures 文件：3 个测试	●●●○○ 3/5	○○○○○ 0/5	5/5	消除 改后读清单，不再比对
对照	同一文件，无关 issue	○○○○○ 0/5	○○○○○ 0/5	0/5	无新增
✗ 改动没有覆盖会话实际遇到的实例					
Sphinx	known-failures 文件 + AGENTS.md	●●●●● 5/5	●●●●● 5/5	5/5	未减少 清单只列 2 个失败，遇到的是别的测试
对照	同一文件，无关 issue	○○○○○ 0/5	●○○○○ 1/5	5/5	+1 次比对
Sphinx	known-failures 文件 + CLAUDE.md	●●●●● 5/5	●●●●● 5/5	5/5	未减少
对照	同一文件，无关 issue	○○○○○ 0/5	●●●●○ 4/5	5/5	增加工作 清单称“未列出的失败都是你改出来的”
SymPy	两个包的 known-failures + 比对脚本	●●●●○ 4/5	●●●● 4/4	4/4	未减少 会话测的是别的包
pytest	known-failures：同一文件 3 个失败	●●●●● 5/5	●●●●● 5/5	5/5	未减少 失败出现在别的文件
Astropy	闰秒缓存修复插件 + 比对脚本	●●●●● 5/5	●●○○○ 2/5	5/5	减轻 无会话遇到闰秒失败；列表外的 NumPy 失败被报成新失败

点的排列只表示计数，不对应具体会话编号。复现标签由一作在不显示 harness 标识的情况下标注，与两名 LLM 标注者在 120 条轨迹中 119 条一致（κ = 0.98）；但轨迹里能看到读取迁移文件，标注并非完全盲。

04 · RQ1 /retro 提了什么：多数有据，lint 常是“惯例”



125 条中 101 条基于会话事件，19 条“先验”，5 条关联弱。**19 条先验提议里 18 条是 lint**（技能说明把“没有 pre-commit / CI 检查”本身算作问题）。38 条可执行检查中，按写好的样子只有 **9 条**能在任务环境里报出失败，其余缺工具或是从不触发的 commit hook。

两个 LLM 编码者按同一码本独立编码（触发码 κ 0.67、机制码 κ 0.84），一作裁决分歧；这不是人类评分者信度。

05 代价：动作少了，任务没变好，时间可能更长

- **≤ 1** 每个事件中两种 harness 的任务成功次数最多差 1 个会话——没有一致的任务成功率收益（作者也不把动作减少等同任务改善）
- **88 → 156 s** SymPy 那条 CLAUDE.md 消除了失败的 pytest 调用，但捆绑的第二行让每个改后会话都跑 CI lint，中位时长上升
- **82 → 245 s** Sphinx 对照任务：不完整的失败清单诱发了它本想防止的基线比对（0/5 → 4/5）
- **−5 至 −28 s** 另外 4 个有信息的事件中位时长下降；SymPy、pytest 两个未覆盖事件时长上升

一条改动可同时含命令指针和 lint 要求，时长变化不能归给单一成分；作者建议后续逐个成分评估。

06 限制（原文 §V 与本项目核对）

- **选题偏向会复现的问题**：后续 issue 是挑来复现目标问题的，第二批 20 个会话也非随机（专挑环境出过问题的仓库）；结果不代表任意后续任务上的帮助频率。
- **样本小、分组在后**：每种 harness 5 个会话、7 个有信息事件；“是否覆盖”是标注后分的组，改动之间同时差好几个属性，不能据此确立覆盖就是原因。
- **不是自主回路**：提议由代理生成，但挑选实施哪条、选后续 issue、标注结果都由作者完成；在原容器里实施，改后文件可能带有提议文本以外的信息。
- **范围窄**：一个代理、一个版本的 /retro、只涉及环境类问题。

评审意见 本项目读法

能带走的判断： harness 自演化的瓶颈不在“生成并实施改动”，而在之后：要逐条确认目标问题有没有出现、代理有没有查阅、动作有没有变，并能修订或撤回没用的改动。仓库级的测试命令可以跨 issue 复用；“某个环境里失败过的具体测试清单”容易覆盖不到下一次遇到的失败，还可能误导验证。

问题档位等字段是本项目的阅读判断，不是审稿结论。